



Technische Hochschule
Ingolstadt

Principles of Autonomy & Decision Making

Final Presentation

Kartik Balasaheb Kapse
(kak5029)

Date :- 01/07/2024



Content:

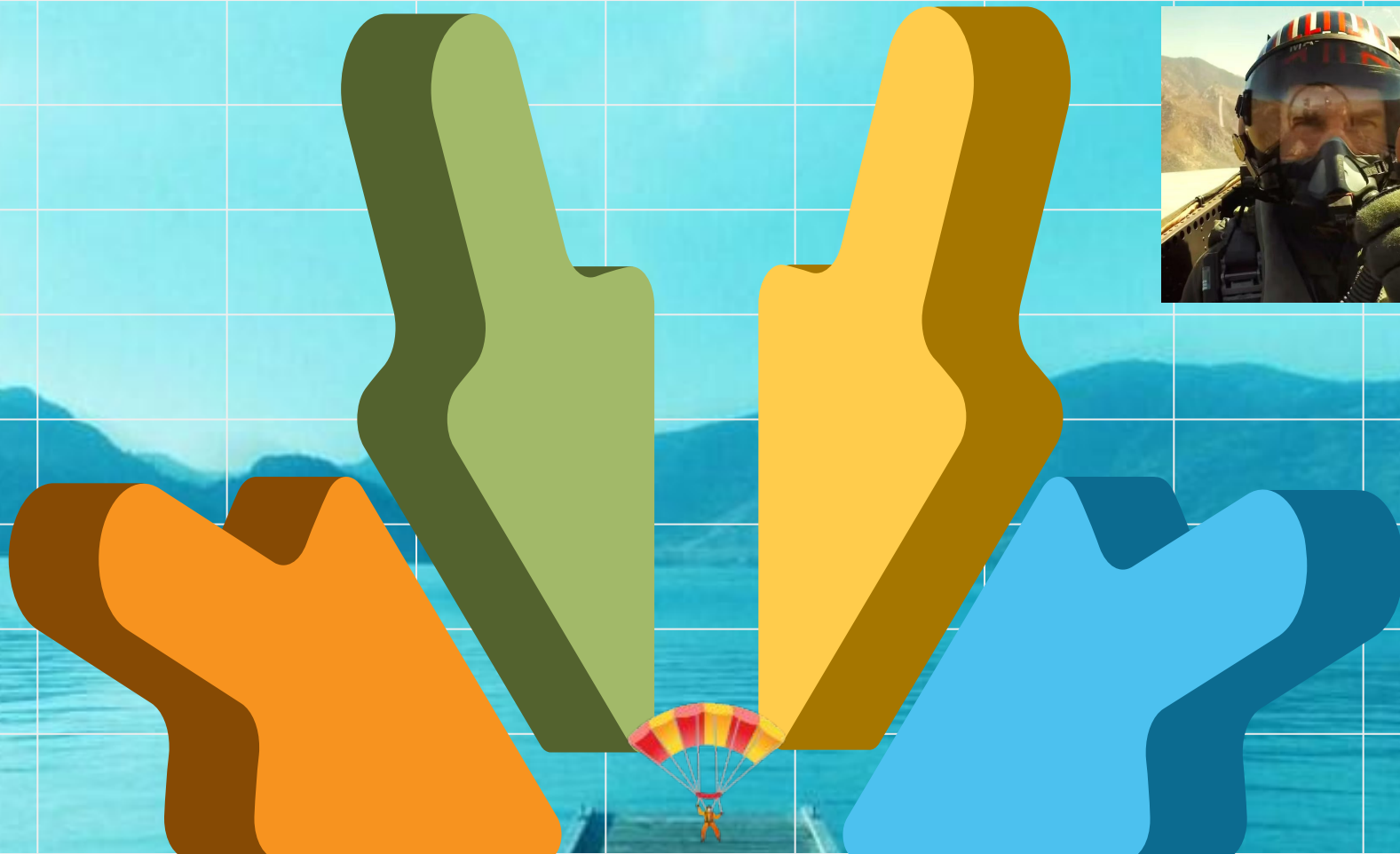
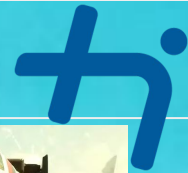
- **Overview of the Environment Parameters**
 - **Introduction to Agent:** MAVERICK
 - **States:** $S = \{S_{0,0}, S_{0,1}, S_{0,2}, S_{0,3}, S_{0,4}, \dots, S_{8,7}, S_{8,8}\}$
 - **Actions:** Move right, left, down, or lift.
- **Epsilon-Greedy Approach:** Balances exploration and exploitation, with decreasing epsilon to favour learned actions.
- **Reward Structure:** Based on interactions, like collecting treasures or hitting obstacles.
- **Results**
- **Reference**



Meet our agent

MAVERICK

Introduction to environment:



Hell State:







Reward Structure:



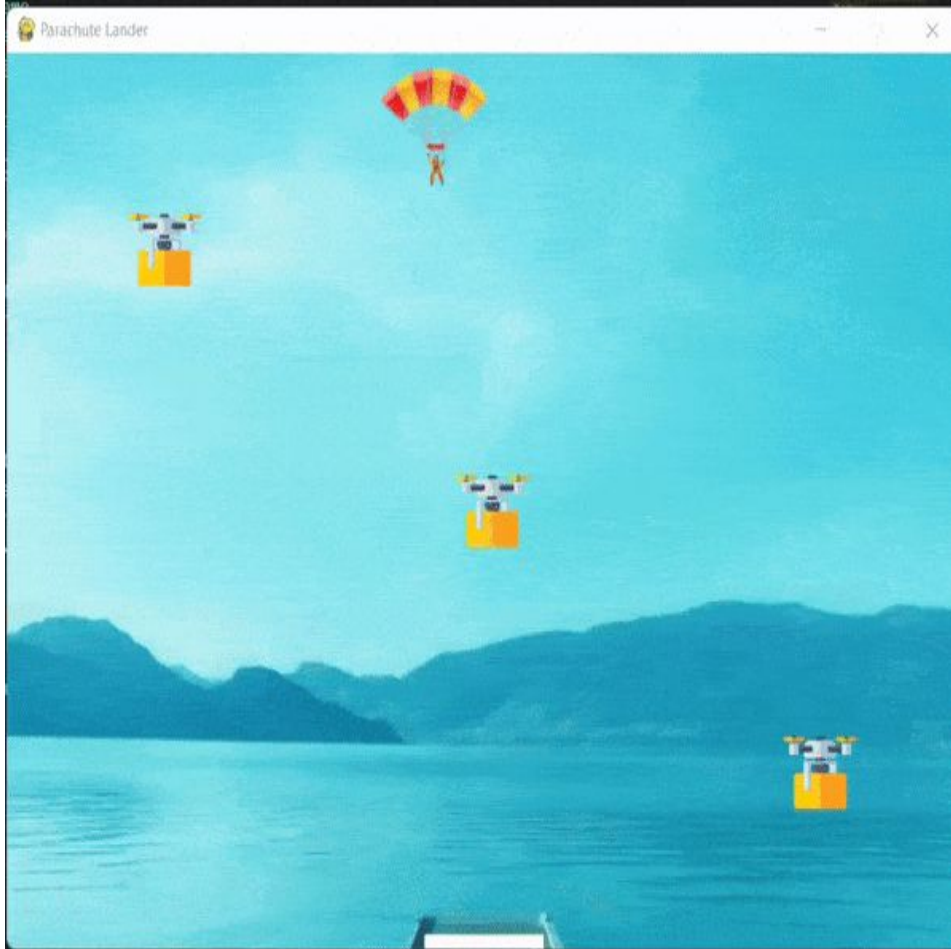
1st Hit = -3.33
2nd Hit = -6.66
3rd Hit = 0 → done = True → reset()

```
# Check if the agent has reached the goal
if np.array_equal(self.state, self.goal):
    self.reward += 10
    self.done = True
elif any(np.array_equal(self.state, each_hell) for each_hell in self.hell_states):
    self.hell_hit_count += 1
    if self.hell_hit_count == 1:
        self.reward += -3.33
    elif self.hell_hit_count == 2:
        self.reward += -6.66
    elif self.hell_hit_count == 3:
        self.reward = 0 # Set the reward to 0 after hitting the third hell state
        self.done = True
else:
    self.reward += -0.05
    self.done = False

def reset(self):
    self.state = np.array([0, 4])
    self.done = False
    self.reward = 0
    self.hell_hit_count = 0 # Reset the hell hit count
    self.info["Distance to goal"] = np.sqrt(
        (self.state[0] - self.goal[0])**2 +
        (self.state[1] - self.goal[1])**2
    )
    return self.state, self.info
```




Overview of the environment:



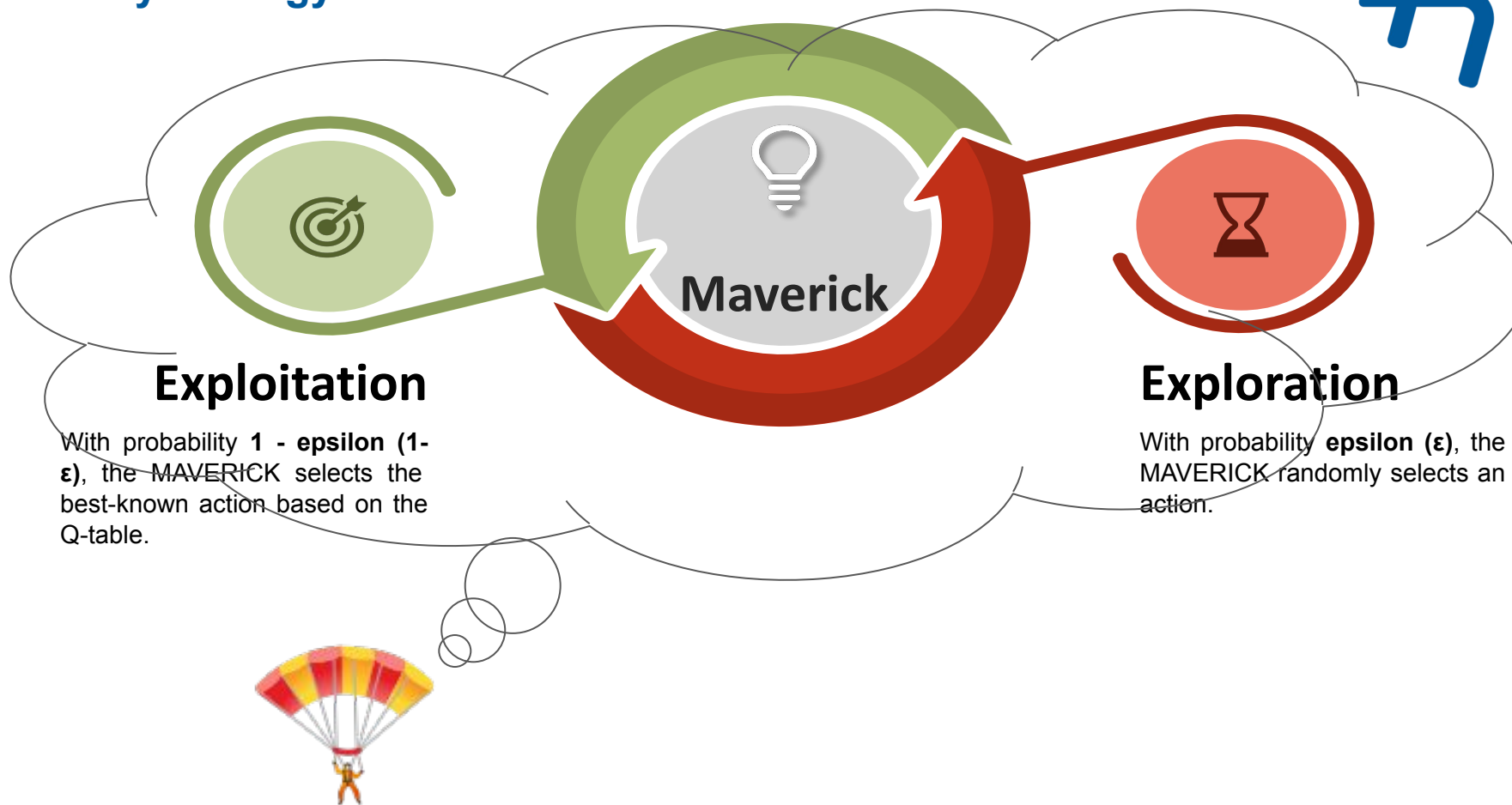
Assignment 1: Grid 800x600



Assignment 3: Grid 9x9



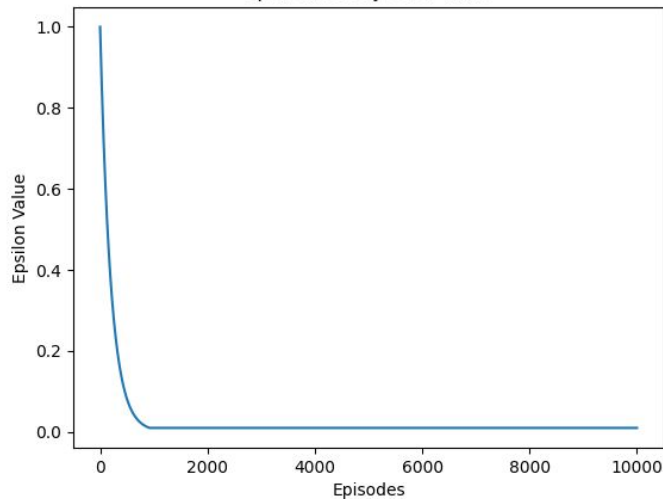
Epsilon decay strategy:



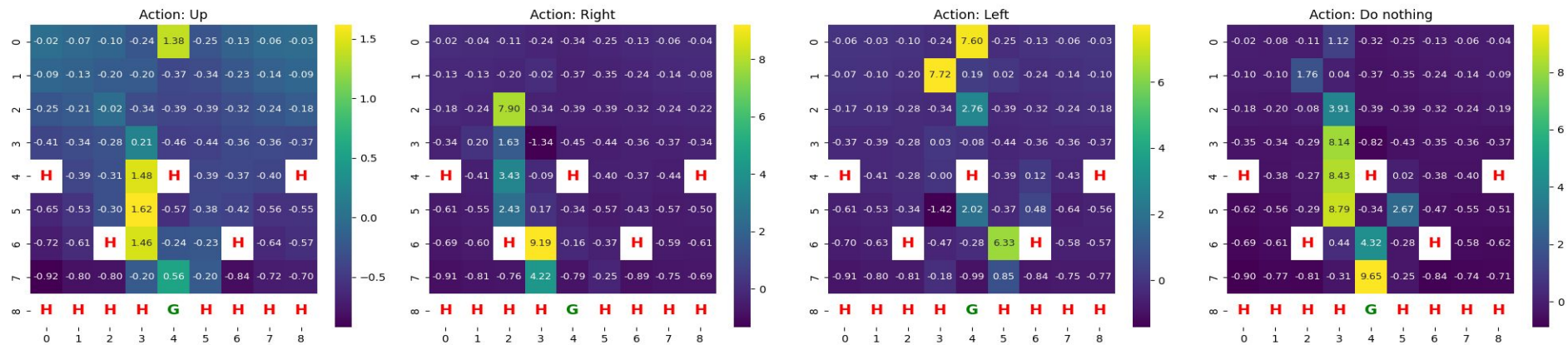


Epsilon decay strategy:

Epsilon Decay Over Time



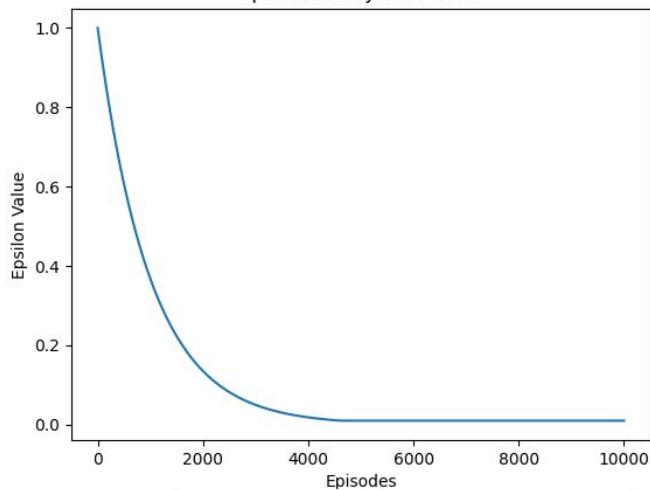
```
1 import matplotlib.pyplot as plt
2
3 learning_rate = 0.01
4 gamma = 0.99
5 epsilon = 1.0
6 epsilon_min = 0.01
7 epsilon_decay = 0.995
8 no_episodes = 10000
9
10 epsilon_values = []
11
12 for episode in range(no_episodes):
13     epsilon_values.append(epsilon)
14     epsilon = max(epsilon_min, epsilon * epsilon_decay)
15
16 plt.plot(epsilon_values)
17 plt.title("Epsilon Decay Over Time")
18 plt.xlabel("Episodes")
19 plt.ylabel("Epsilon Value")
20 plt.show()
```



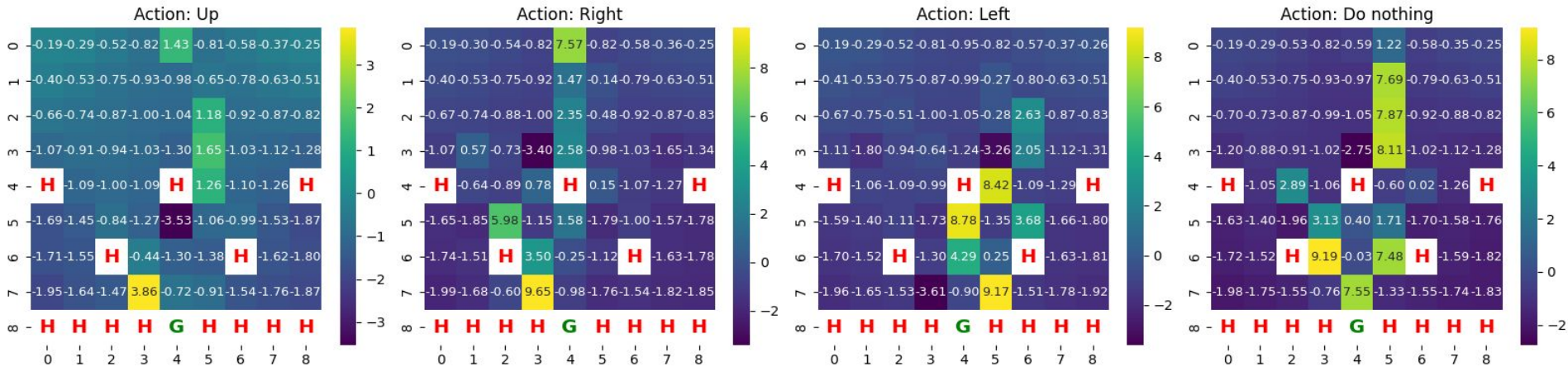


Epsilon decay strategy:

Epsilon Decay Over Time



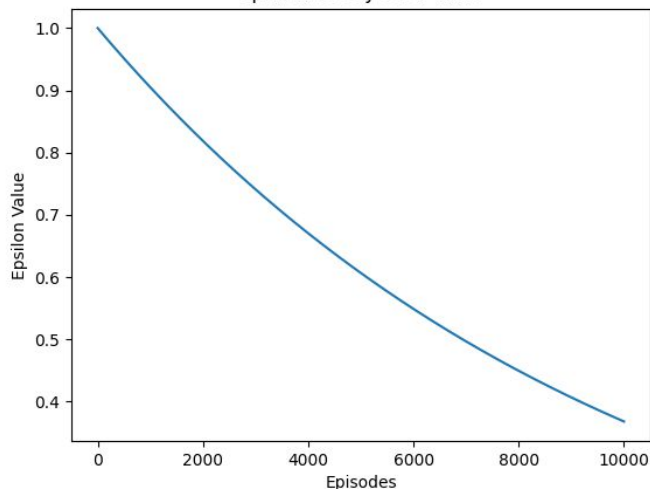
```
1 import matplotlib.pyplot as plt
2
3 learning_rate = 0.01
4 gamma = 0.99
5 epsilon = 1.0
6 epsilon_min = 0.01
7 epsilon_decay = 0.999
8 no_episodes = 10000
9
10 epsilon_values = []
11
12 for episode in range(no_episodes):
13     epsilon_values.append(epsilon)
14     epsilon = max(epsilon_min, epsilon * epsilon_decay)
15
16 plt.plot(epsilon_values)
17 plt.title("Epsilon Decay Over Time")
18 plt.xlabel("Episodes")
19 plt.ylabel("Epsilon Value")
20 plt.show()
```



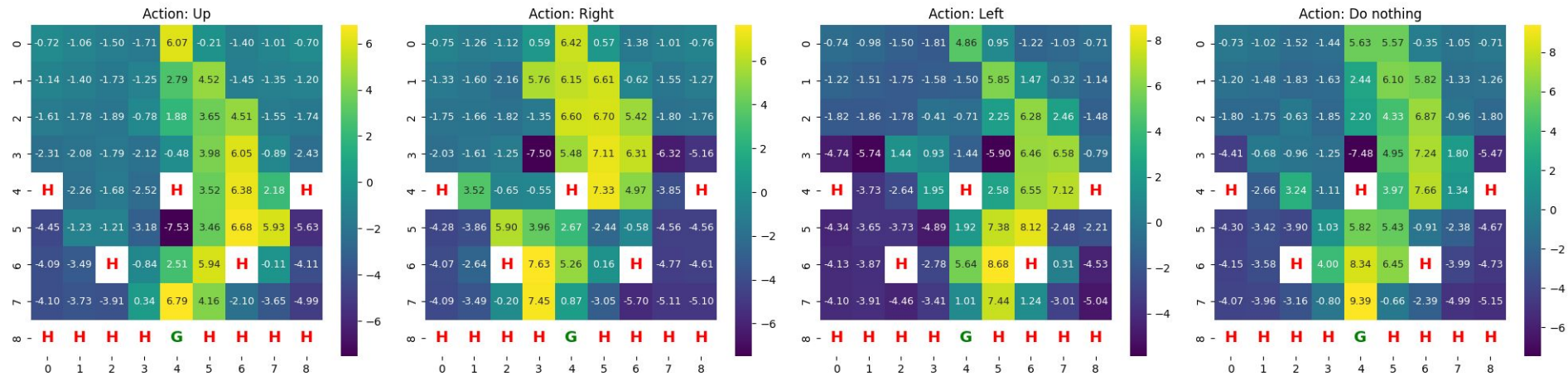


Epsilon decay strategy:

Epsilon Decay Over Time

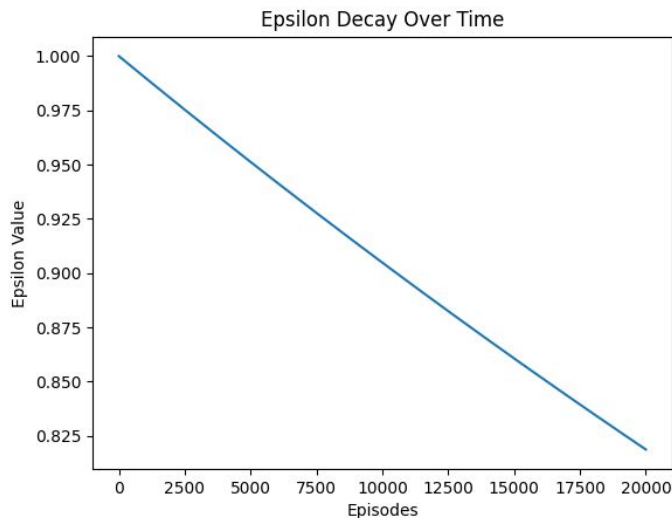


```
1 import matplotlib.pyplot as plt
2
3 learning_rate = 0.01
4 gamma = 0.99
5 epsilon = 1.0
6 epsilon_min = 0.001
7 epsilon_decay = 0.9999
8 no_episodes = 10000
9
10 epsilon_values = []
11
12 for episode in range(no_episodes):
13     epsilon_values.append(epsilon)
14     epsilon = max(epsilon_min, epsilon * epsilon_decay)
15
16 plt.plot(epsilon_values)
17 plt.title("Epsilon Decay Over Time")
18 plt.xlabel("Episodes")
19 plt.ylabel("Epsilon Value")
20 plt.show()
```

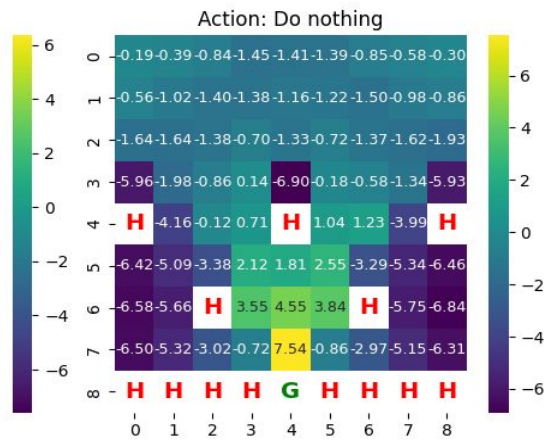
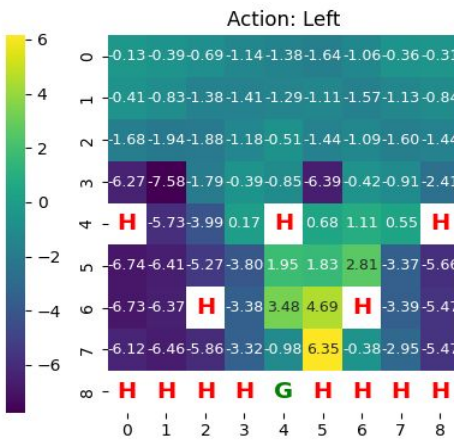
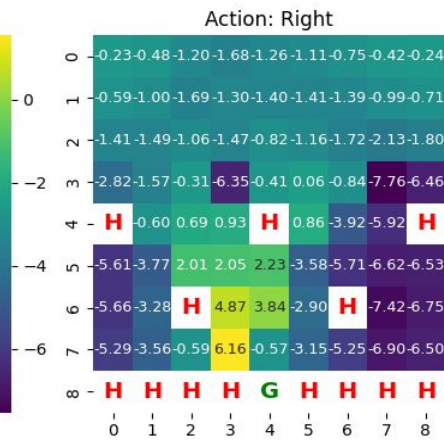
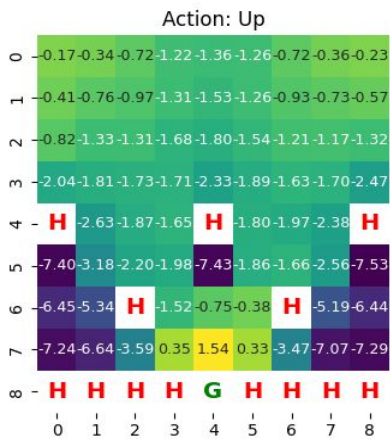




Epsilon decay strategy:



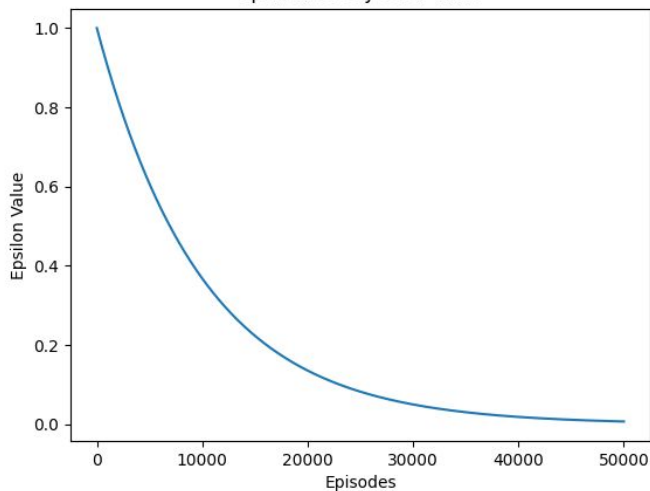
```
1 import matplotlib.pyplot as plt
2
3 learning_rate = 0.01
4 gamma = 0.99
5 epsilon = 1.0
6 epsilon_min = 0.0001
7 epsilon_decay = 0.99999
8 no_episodes = 20000
9
10 epsilon_values = []
11
12 for episode in range(no_episodes):
13     epsilon_values.append(epsilon)
14     epsilon = max(epsilon_min, epsilon * epsilon_decay)
15
16 plt.plot(epsilon_values)
17 plt.title("Epsilon Decay Over Time")
18 plt.xlabel("Episodes")
19 plt.ylabel("Epsilon Value")
20 plt.show()
```



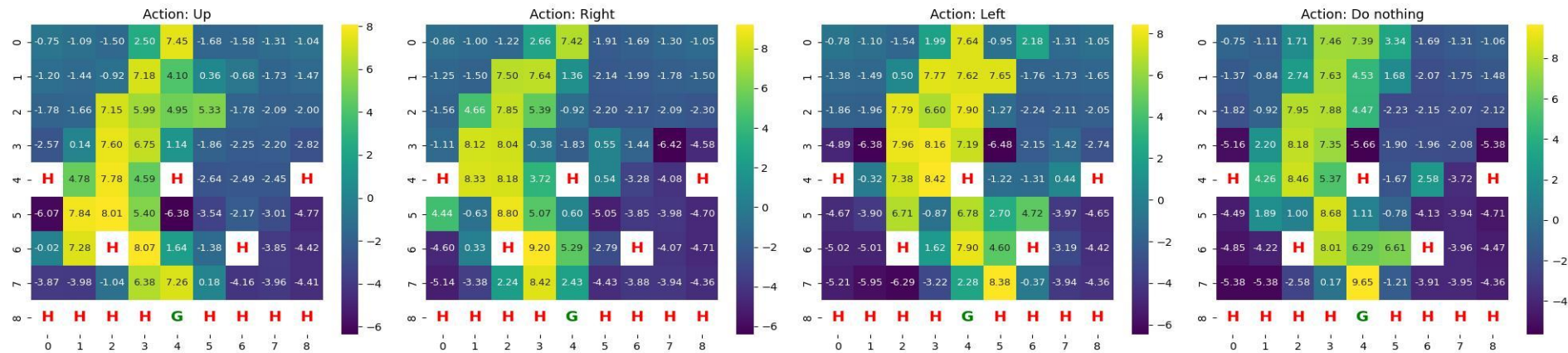


Epsilon decay strategy:

Epsilon Decay Over Time



```
1 import matplotlib.pyplot as plt
2
3 learning_rate = 0.01
4 gamma = 0.99
5 epsilon = 1.0
6 epsilon_min = 0.0001
7 epsilon_decay = 0.9999
8 no_episodes = 50000
9
10 epsilon_values = []
11
12 for episode in range(no_episodes):
13     epsilon_values.append(epsilon)
14     epsilon = max(epsilon_min, epsilon * epsilon_decay)
15
16 plt.plot(epsilon_values)
17 plt.title("Epsilon Decay Over Time")
18 plt.xlabel("Episodes")
19 plt.ylabel("Epsilon Value")
20 plt.show()
```



S(0,0)	S(0,1)	.	.	Experiments:			S(0,599)	S(0,600)
S(1,0)	.	.	.	.	.	.	.	S(1,600)
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.
S(799,0)	.	.	.	.	.	.	.	S(799,600)
S(800,0)	.	.	.	.	.	.	S(800,599)	S(800,600)

MAVERICK lands and MISSION ACCOMPLISHED



References:



- Lectures and Moodle materials (SS-2024) Prof. Dr. rer. nat. Lenz Belzner, Chidvilas Karpenahalli Ramakrishna, Adithya Mohan, Zahra Zeinaly.
- Stanford CS234 Reinforcement Learning, stanford online, YouTube
- Stanford CS229 Machine Learning (Autumn 2018), YouTube
- Stanford CS221: AI (Autumn 2019), Youtube



**Thank You
Q&A
Feedback**